X-Bucket: A Family of Algorithms for Adaptive Resource Allocation in Dynamic Workflows

Thanh Son Phung and Douglas Thain

***Abstract*—Dynamic workflow systems enable local applications to scale out by decomposing their computational needs into task graphs on-the-fly and executing them on large clusters. However, this dynamic task generation raises a challenging resource allocation problem: upon scheduling, a task needs to be allocated some amount of resources, yet its resource consumption is only known after its execution. This uncertainty, combined with workflows' large scale and internal and external stochastic behaviors, can result in huge resource waste when resource allocation decisions are manually or statically made in advance. To tackle this problem, we introduce *X-Bucket*, a family of four *general-purpose, prior-free, online*, and *robust* algorithms for adaptive resource allocation in workflow systems. Specifically, algorithms in *X-Bucket* share the approach of optimally separating collected resource reports of completed tasks into different buckets to minimize resource waste, while they differ in the way buckets are discovered and used for resource allocation prediction of subsequent tasks. We implement *X-Bucket* algorithms in WorkQueue and TaskVine, two mature dynamic workflow systems, and show through our evaluation on eight diverse workflows that the *X-Bucket* family consistently outperform five alternative algorithms in resource allocation efficiency, with the best algorithm incurring at most 3.9 ms per prediction in the steady state.**

***Index Terms*—Workflow Management Systems, Resource Management, Predictive Models**

I. Introduction

Dynamic workflow management systems [1]–[3] are prevalent in large-scale scientific computing on HPC clusters due to its high degree of flexibility in formulating and solving computational problems across diverse domains [4]–[7] with an intuitive execution and scaling model. Particularly, computations are written in functions in a high-level language, which are packaged into portable tasks with their definitions and software dependencies for execution on remote nodes. Task graphs are also no longer required to be specified in advance as tasks' inter-dependencies are inferred by functions' invocation patterns. Additionally, dynamic workflows require no static resource provisioning as opposed to traditional monolithic or MPI-based applications. The workflow computation rate can instead scale linearly with the amount of available resources and typically varies based on the user's preference and the cluster's current resource availability.

However, the problem of resource management, specifically resource allocation, becomes much harder at the task scheduling time due to the dynamic nature of these systems. It is crucial to optimally allocate a certain amount of cores, memory, and disk to a given task to prevent resource waste and help the scheduler component make quality task-to-node scheduling decisions [8]. Unfortunately, resource allocation to tasks is an online problem: **the optimal resource allocation for a task is its peak resource consumption, but this information is unknown at scheduling time and only visible after the task's execution**. Manually assigning resources to tasks before a workflow run is not realistic or scalable: non-technical users are unlikely to possess expert-level knowledge in correlating tasks with their resource requirements, and dedicated workflow developers simply cannot follow and assign resources to every task in every workflow any user has ever created.

Alternatively, users can have an educated guess and set a static upper-bound resource allocation for all tasks in advance. While this solution can work for simple deterministic workflows with homogeneous resource consumption patterns, large-scale dynamic workflows tend to have much more stochastic behaviors. Tasks are usually specialized for different computational purposes and divided into different workflow phases, leading to a moving resource consumption distribution over time. Workflows may also evolve over time to incorporate additional task logic, updates to underlying software dependencies and runtimes, or the arrival of new datasets, prompting a new resource allocation guess per change. Tasks themselves can be inherently stochastic and exhibit different resource requirements over consecutive executions. These problems, combined with the typical scale of thousands of tasks in workflows, can result in massive resource waste even with a well-informed guess.

The nature of software deployment of dynamic workflow systems brings another operational consideration. These systems are typically packaged as standalone frameworks or libraries where users first download and install them locally, then write new workflow-based applications via exposed APIs. Due to this standalone nature and the diversity of top-level applications, a candidate solution for the resource allocation problem in dynamic workflow systems must work with *every* possible workflow expression. That is, the solution's primitives must be denominated by the most generic workflow abstraction to preserve the general applicability of workflow systems in diverse scientific domains.

Tackling all these problems requires a data-driven approach that adaptively predicts resource allocation for subsequent tasks based on collected data about completed tasks' resource consumption. A complete solution - algorithm *X* - must follow the following design goals:

1) **General-purpose**: *X* must be able to run generically with any workflow, and thus must not rely on workflow- or task-specific features.

Thanh Son Phung and Douglas Thain are with the University of Notre Dame (email: tphung@nd.edu; dthain@nd.edu).

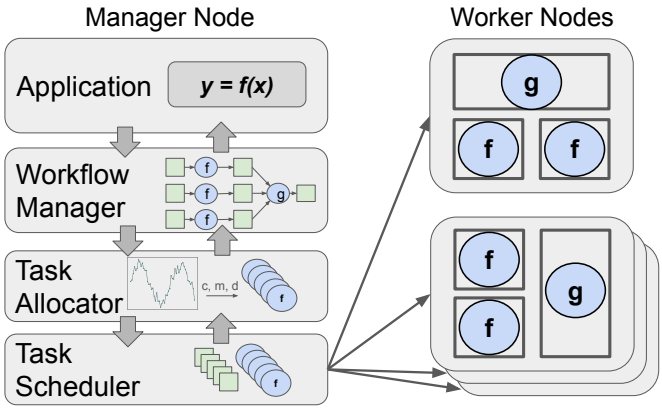


Fig. 1. Dynamic Workflow System Architecture. An application first defines computations for remote execution and passes them to the workflow manager, which translates them into tasks and constructs a task graph on-the-fly. Ready tasks are allocated a certain amount of resources before being scheduled for remote execution on worker nodes. Once tasks complete their executions, resource reports and results are sent back to the task allocator and the application, respectively.

2) **Prior-free**: *X* must not rely on information of past runs of a given workflow (e.g., previous traces, logs) to predict resource allocations for the current run. Related works [9]–[11] utilize machine learning techniques to predict customized resource allocations for specific applications. These techniques however rely on past training data, break the general applicability of workflow systems, tend to overfit, are susceptible to workflow stochasticity and incompatible with new workflows that have not been run before.
3) **Online**: *X* must operate in an online manner due to the dynamic nature of workflows. That is, *X* must be able to utilize collected information of completed tasks' resource consumption in predicting resource allocations for subsequent tasks as the workflow runs.
4) **Robust**: *X* must outperform related works [8], [12] under a diverse set of workflow resource consumption and stochastic behaviors.

To satisfy this list of design goals, we introduce four predictive algorithms in the *X-Bucket* family for adaptive resource allocation, namely *Incremental Greedy Bucketing (IG-Bucket), Incremental Exhaustive Bucketing (IE-Bucket), Probabilistic Greedy Bucketing (PG-Bucket)*, and *Probabilistic Exhaustive Bucketing (PE-Bucket)*. All algorithms attempt to minimize the expected resource waste of all tasks in the workflow by optimally separating collected resource consumption from completed tasks into different resource buckets, while differ in the way these buckets are discovered (i.e., *Exhaustive* versus *Greedy*) and used (i.e., *Probabilistic* versus *Incremental*) for predicting resource allocation of subsequent tasks. They are carefully designed to be *general-purpose* (no workflow- or task-specific feature is used), *prior-free* (no history of past workflow runs is collected: only the history of the current run is used), *online* (resource prediction is on the critical path of task scheduling and immediate upon request), and *robust* (the prediction value is automatically updated as workflows change their behaviors at runtime.)

We implement the *X-Bucket* family in two mature workflow systems, WorkQueue [13] and TaskVine [14], and evaluate them on three production workflows - ColmenaXTB [15], TopEFT [16], and LNNI [17] (see Figure 2) - and five synthetic workflows following five diverse resource distributions (*Normal, Uniform, Bimodal, Phasing Trimodal,* and *Exponential* - see Figure 6). All workflows are run with 20 to 50 workers deployed opportunistically on the local HTCondor [18] cluster. Our results show that algorithms in the *X-Bucket* family significantly outperform five alternative algorithms in resource savings and allocation efficiency, reaching as high as 92.3% average resource efficiency and 92.1% absolute resource efficiency, with the best algorithm incurring at most 3.9 ms of prediction overhead per resource allocation request. Note that algorithms in the *X-Bucket* family are designed to use the most generic primitives of a workflow system, requiring only the resource consumption reports of completed tasks during a workflow run. We thus believe these algorithms can be readily implemented in other workflow systems beyond WorkQueue and TaskVine. This paper is an extension from a previous conference paper [19], contributing (1) two new algorithms, *IG-Bucket* and *IE-Bucket*, to fully form the *X-Bucket* family, (2) a new metric along with associated analyses and results tracking the resource efficiency of all algorithms in the long run, (3) a new production workflow, LNNI, to cover the domain of machine learning inference, (4) a detailed analysis determining the best algorithm in the family, (5) a detailed description and analysis of *Bucket Composer*, a new strategy running on top of the *X-Bucket* family that merges separate predictions in cores, memory, and disk into one schedulable resource allocation prediction with practical considerations, and (6) the integration of the *X-Bucket* family in TaskVine.

II. Problem Framework

A. Background

The general architecture of a dynamic workflow system is shown in Figure 1 where the control plane resides on the manager node and the execution plane resides on remote worker nodes. On the manager node, the application first defines a large number of computations as functions and passes them via invocations to the workflow manager. The workflow manager takes care of packaging them into tasks by marshalling their arguments, detecting software dependencies, wrapping error handling code, etc., and resolves the inter-task dependencies into a dynamic task graph. Ready tasks are sent to the task allocator for resource allocation (e.g., cores, memory, disk, GPUs) before they arrive at the scheduler. The scheduler then maps tasks with a certain resource allocation into available workers for remote execution, where each task is run in a file system and resource sandbox. Upon task completion, workers send back the measured resource consumption of tasks to the task allocator for data collection purposes, and tasks' results to the application. **Note that the resource allocation problem (i.e., how much resources to give to a task) is distinct from the task scheduling problem (i.e., which node to assign a task to), and a solution to the former operates after**

ready tasks are detected and before tasks are scheduled for remote execution.

B. Definitions and Assumptions

To work with the most generic workflow abstraction, our problem space only involves two entities: *Task* and *Allocation*. A task *T* (c, m, d, t) is an isolated executable program that, when executed, consumes at most *c* cores, *m* MBs of memory, *d* MBs of disk in *t* seconds. An allocation *A* (c_a, m_a, d_a) declares the amount of resource requirements for a task to the scheduler, such that the worker shall provision c_a cores, m_a MBs of memory, and d_a MBs of disk for the task to execute. In practice, many batch and workflow management systems monitor tasks' resource consumption over time and kill them the moment they detect resource over-consumption. We will adopt this reasonable convention and introduce the following set of assumptions:

1) **At the allocation time, the optimal amount of resources to allocate a task is unknown**.
2) **Resource allocation is on the critical path of task scheduling: a task must have a resource allocation before being scheduled for remote execution.**
3) **Over the course of a task's execution, it can only consume resources up to the amount of its allocation.**
4) **Upon the detection of resource over-consumption, the task's execution is terminated and its existing resource report is sent to the resource allocator for subsequent retries with bigger allocations.**

A task then executes successfully only if $c \leq c_a$, $m \leq m_a$, $d \leq d_a$. The core problem is the potential resource waste stemming from assumption (1): a large allocation risks a large resource waste, while an inadequate allocation wastes its resources and triggers task failures and retries.

C. Goals and Metrics

The goal of any predictive resource allocation algorithm is zero resource waste and 100% resource efficiency - the performance of an oracle algorithm. We will now define all relevant metrics used in the paper, assuming that for a given resource *R*, a task *T* is successfully allocated with *a* units of resources and consumes at most *c* units during its execution over t seconds.

Resource Waste: We first break down resource waste of a task into two types: *Internal Fragmentation* and *Failed Allocation*.

1) *Internal Fragmentation*: This waste type tracks the unused resources in a successful allocation of a task, and is defined to be the difference over time between *T*'s peak resource consumption and its allocation, i.e., $t \cdot (a - c)$, provided that $c \leq a$.
2) *Failed Allocation*: Assumption 4 in Section II-B dictates that when *T* over-consumes its resource allocation, it must be retried with a larger allocation. This implies that the current task try does not complete any work and incurs a badput resource waste. *Failed Allocation* is then defined to be $\Sigma_{i=1}^{k}(a_i \cdot t_i)$, where k is the number of failed allocation tries and each pair of (a_i, t_i) is the amount of allocated resource and execution time at the ith try.

The total resource waste of *T* is the sum of two resource waste types:

$$ResourceWaste(T) = t \cdot (a - c) + \Sigma_{i=1}^{k}(a_i \cdot t_i)$$

The resource waste of *T* is optimal when both waste types are minimized to 0. That is, *T* is allocated exactly once, and its resource allocation perfectly matches its resource consumption.

Absolute Workflow Efficiency - *AWE*: *AWE* attempts to track the absolute efficiency in resource usage of a workflow for a given type of resource *R*. To avoid a clutter of symbols, we first define the resource consumption and total resource allocation of task *i* as $C(T_i)$ and $A(T_i)$, where

$$C(T_i) = c_i \cdot t_i$$

$$A(T_i) = a_i \cdot t_i + \Sigma_{j=1}^{k_i}(a_{ij} \cdot t_{ij})$$

AWE is then defined to be

$$AWE(\{T_i\}_1^n) = \frac{\Sigma_{i=1}^{n} C(T_i)}{\Sigma_{i=1}^{n} A(T_i)},$$

where $\{T_i\}_1^n$ is a sequence of tasks from T_1 to T_n in workflow W. *AWE*'s numerator tracks the total consumption of all tasks in W, while the denominator tracks the total allocation of all tasks in W. Thus, this metric considers the entire workflow as a whole and measures the resource usage for "goodput" computations via the ratio of total useful resource consumption over the total resource allocation. A workflow is then allocated optimally iff its *AWE* is equal to 1, i.e., it consumes all of its allocated resources.

Average Task Efficiency - *ATE*: While *AWE* tracks the true resource efficiency of a workflow, *ATE* tracks the expected resource efficiency of workflow over a long run by averaging the resource efficiency of individual tasks for a given resource type. Using the same notations of $C(T_i)$ and $A(T_i)$ as above, *ATE* is then defined to be

$$ATE(\{T_i\}_1^n) = \frac{1}{n}\Sigma_{i=1}^{n}\frac{C(T_i)}{A(T_i)}$$

Since the *X-Bucket* family and alternative algorithms must be prior-free and can only use resource reports of completed tasks in the current workflow run, they must pay some exploration cost to collect the very first reports. As opposed to *AWE* fully accounting for this exploration cost, *ATE* smoothens this cost out and allows us to measure an algorithm's quality of resource prediction in the steady state.

Composite Resource Efficiency - *CRE*: While other efficiency metrics tracked during a workflow run are per resource type (e.g., cores, memory, disk), *CRE* attempts to aggregate all these metrics into a singular resource-agnostic value, reflecting the resource efficiency of the workflow regardless of the resource type. Ensuring that the metric aggregation process is fair and reflecting the true resource efficiency of workflows is important. We thus introduce two sub-metrics in *CRE*: *homogeneous CRE* (or *H-CRE*), and *power-denominated CRE*

(or *P-CRE*). Assume that a given workflow has resource efficiency values of cores, memory, and disk as e_c, e_m, and e_d, *H-CRE* is defined as the equal-weight geometric mean of these values.

$$H\text{-}CRE(e_c, e_m, e_d) = \sqrt[3]{e_c \cdot e_m \cdot e_d}$$

This metric then computes the average resource efficiency of a workflow given that all resource types are equally important and thus treated equally fairly. This set of equal weights might not reflect how the resource efficiency of different resource types is valued however (e.g., a user running a CPU-intensive workflow might care more about the core efficiency of tasks and thus shift more weights to e_c.) *P-CRE* then attempts to quantify the resource-agnostic efficiency using the weight set of [0.65, 0.30, 0.05] for cores, memory, and disk, respectively, following the chosen weights for power efficiency as standardized in [20], [21] (note that possible correlations in power consumption between resource types are built into these weights). Note that we restrict *H-CRE* and *P-CRE* to only aggregate *AWE* results of cores, memory, and disk to ensure true resource efficiency reporting. *P-CRE* is then defined as a weighted geometric mean:

$$P\text{-}CRE(e_c, e_m, e_d) = e_c^{0.65} \cdot e_m^{0.3} \cdot e_d^{0.05}$$

Moreover, note that all metrics are *independent* of the number of workers available to a given dynamic workflow, allowing the underlying system to freely scale out or in its capacity without affecting the efficiency metrics.

D. Additional Problems and Solution Designs

In addition to the inherent uncertainty as outlined in Section II-B, the resource allocation problem is further complicated by stochastic behaviors of workflows, which can be divided into two categories: *Internal Stochasticity* and *External Stochasticity*.

Internal Stochasticity: This category captures random elements that are uncontrollable within the execution of a workflow, such as:

1) *Arbitrary ordering of task execution*: Tasks are defined and submitted sequentially to workflow systems, but potentially executed in an arbitrary order due to a variety of factors (e.g., data and result dependencies between tasks, priority of tasks, data locality on workers' disk, available capacity of workers).
2) *Task specialization*: Complex workflows frequently define tasks of different natures: tasks thus can be core-, memory-, or I/O-intensive depending on their computational purposes.
3) *Arbitrary structure of workflows*: Workflows are often organized into phases of tasks for coordination and computational purposes, each of which possibly has a different resource consumption profile depending on the current phase.

Due to the stated characteristics, complex workflows are expected to exhibit an *arbitrary moving resource consumption distribution* over time depending on the phase and groups of tasks currently executed.

External Stochasticity: This category attempts to enumerate random and/or uncontrollable elements that happen between executions of the same workflow, such as:

1) *Current system state*: Resources on shared premises (e.g., compute cluster, storage servers) are subject to arbitrary usage from other users, which in turn influences the amount of compute or data bandwidth available to a workflow between different runs, and further contributes to *Internal Stochasticity* [22].
2) *Evolution of workflows*: Workflows evolve and thus behave differently between executions to accommodate for software dependency updates, arrival of new datasets, or changes in the workflow logic to task or phase configurations.
3) *Inherent stochasticity of tasks*: Tasks themselves can be stochastic (e.g., Monte-Carlo simulations, SGD-based ML training) and thus consume resources differently between executions.

This combination of inherent uncertainty, practical software deployment of workflow systems, and *Internal* and *External Stochasticity* must be considered when evaluating the practicality of a candidate algorithm *X* to the resource allocation problem. Therefore, we propose that *X* must meet the four following design goals:

1) **General-purpose**: Note that the optimization of a workflow's resource waste concerns only a task's resource consumption and allocation (see Section II-C). This minimal formulation, along with the general applicability requirement of workflow systems, necessitates *X* be as generic as possible and not rely on task- or workflow-specific features.
2) **Prior-free**: Random elements in *External Stochasticity* diminish the value of prior information about workflows as such information are application-specific and susceptible to workflow changes.
3) **Online**: The consequence of *Internal Stochasticity* demands that *X* be an online algorithm, deriving resource allocation predictions for tasks in real time to adapt to the ever-changing resource consumption distribution.
4) **Robust**: Elements in *Internal Stochasticity* require *X* to perform well on arbitrary resource consumption distributions and phase changes during a workflow run.

III. Case Study: ColmenaXTB, TopEFT, and LNNI

We now examine the resource consumption logs of three production workflows, ColmenaXTB, TopEFT, and LNNI, to show that elements of workflow stochasticity are beyond theoretical speculation and indeed appear in and impact production workflows.

A. Overview

All production workflows follow the architecture in Figure 1. Colmena-XTB is a distributed application combining neural network inferences with molecular dynamics analysis to drive

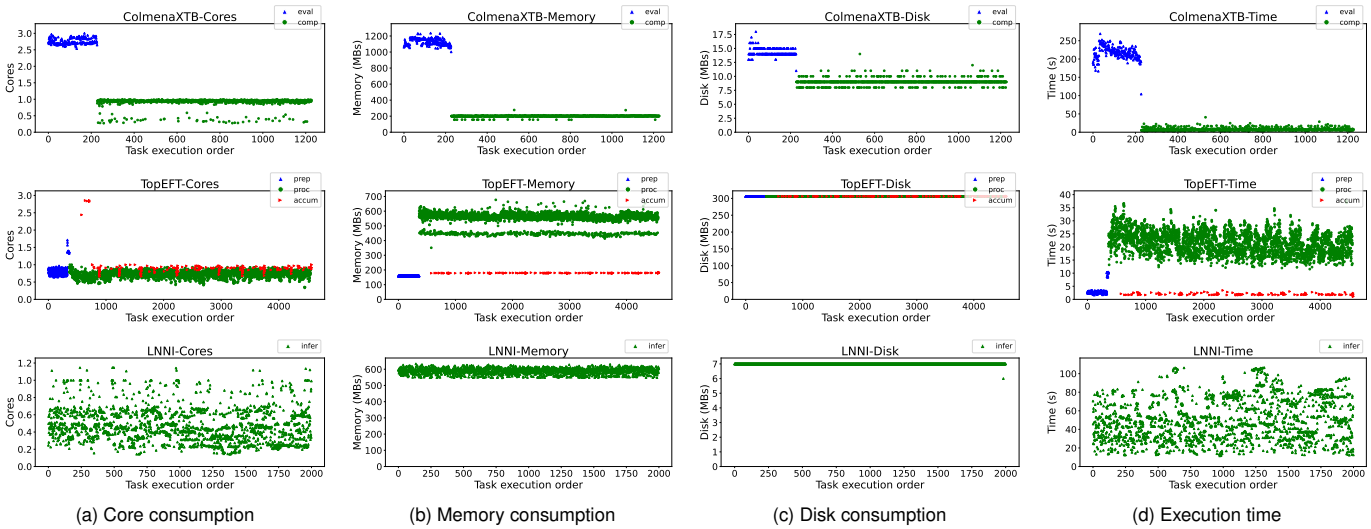


Fig. 2. Resource consumption of tasks in production workflows: ColmenaXTB (top row), TopEFT (middle row), and LNNI (bottom row). Each point represents a task's peak resource consumption. Cores, memory, and disk are defined as the average CPU time, resident set size, and local storage footprint per task.

large campaigns of molecular search and design. It defines two functions: (1) *evaluate_mpnn*, which inputs a list of candidate molecules and outputs a ranking of those molecules by expected energies, and (2) *compute_atomization_energy*, which computes the energy values from top-ranked molecules. ColmenaXTB's software stack includes a suite of distributed execution frameworks, including (1) Colmena [23], a Python library driving the molecular search campaign by carefully and dynamically submitting tasks and examining returned results, (2) Parsl [1], a Python-native workflow manager that constructs a task dependency graph on the fly and send ready tasks to (3) WorkQueue, a dynamic workflow system that handles the allocation, scheduling, and execution of tasks along with result collection and worker management on various underlying batch systems.

TopEFT is a distributed application applying the effective field theory (EFT) to detect new physics by processing a large quantity of events produced by the Large Hadron Collider (LHC). It defines three functions: (1) *preprocessing*, which scans through a list of metadata files to find relevant event datasets, (2) *processing*, which analyzes a given quantity of events, and (3) *accumulating*, which merges processed results into a complex multi-level histogram. Once defined, all functions are passed down to Coffea [24], a high-performance data processing library. Coffea first submits all preprocessing tasks to fetch metadata files and identify relevant event datasets, then logically divides events between processing tasks or partially accumulated results between accumulating tasks. All tasks generated by Coffea are sent to Work Queue for remote execution.

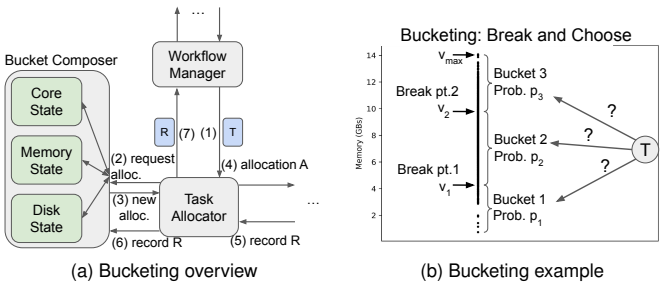
Finally, LNNI is a large-scale neural network inference application running 16 inferences per task on a pretrained ResNet50 [25] model. It defines the inference function using APIs exposed by the TaskVine frontend API. Once the function is invoked, TaskVine converts the invocation into a portable task and sends it to TaskVine backend for remote execution as shown in Figure 1. TaskVine backend behaves similarly to WorkQueue, which manages task execution, result collection, worker coordination, etc.

B. Analysis on Workflows' Resource Consumption

Figure 2 shows the resource consumption of tasks in all production workflows. Each row represents all data collected for a given workflow, and each column represents the resource type, varying from cores, memory, disk, to time. Within each plot, the x-axis shows the order of task submission represented by an incremental ID starting from 0, and the y-axis shows the magnitude of a given resource type. Each point in a plot is thus a given task's peak resource consumption. ColmenaXTB has 228 *evaluate_mpnn* tasks and 1000 *compute_atomization_energy* tasks; TopEFT has 363 *preprocessing* tasks, 3994 *processing* tasks, and 212 *accumulating* tasks; LNNI has 2000 *inference* tasks.

Task specialization: For ColmenaXTB and TopEFT, we can see major differences in resource consumption between tasks of different categories, especially in memory consumption. In ColmenaXTB, while *evaluate_mpnn* tasks consume anywhere from 1 to 1.2 GBs of memory, *compute_atomization_energy* tasks instead only hover around 200 MBs, suggesting that different task categories may need different amount resource allocations. However, in TopEFT, *preprocessing* and *accumulating* tasks nearly use the same amount of memory (around 200 MBs). This implies that tasks of different categories should instead be allocated *independently* from each other, as tasks' categories and their resource consumptions do not necessarily have a mutual relationship.

Arbitrary structure of workflows: We also see that ColmenaXTB and TopEFT have different task phases to reflect different computational purposes. ColmenaXTB first submits *evaluate_mpnn* tasks to retrieve a list of top-ranked molecules, and only then it submits the following *compute_atomization_energy* tasks to process them. This phasing



(a) Bucketing overview (b) Bucketing example

Fig. 3. The bucketing approach in the big picture (a) and in an example (b).

behavior thus accounts for the elevated core and memory consumption of *evaluate_mpnn* tasks in the first phase compared to that of *compute_atomization_energy* in the second phase. TopEFT also shows a similar behavior, with only *preprocessing* tasks running and completing first before *processing* and *accumulating* tasks are able to run, causing a sudden jump in memory consumption from *processing* tasks. Moreover, the memory consumption of *processing* tasks in TopEFT tells a puzzling story, as tasks of the same type can be seemingly separated into two clusters due to their memory consumption (cluster 1 around 580 MBs, and cluster 2 around 450 MBs). The core consumption of TopEFT tasks also shows another aspect of task stochasticity: *outliers*. While most tasks consumes one core or less during their execution, some tasks (possibly receiving more interesting and compute-intensive events) consume as many as three cores, undermining the effectiveness of manual or static resource allocation strategies. Even the seemingly more homogeneous workflow like LNNI shows a huge variance in their core consumption, jumping between 0.2 and 1.2 cores.

This case study on ColmenaXTB, TopEFT, and LNNI shows that several elements of workflows' stochasticity indeed exist, which emphasizes the practical and challenging nature of the resource allocation problem for dynamic workflows. Since the structure and computation formulation of these workflows are quite common, we believe that the stochastic elements are not limited to these workflows, the underlying systems of WorkQueue and TaskVine, but are generalizable to other large-scale and complex production workflows and systems. Therefore, this case study validates the design goals of a candidate algorithm *X* as stated in Section II-D and stresses a practical need for *X* conforming to these goals.

IV. The *X-Bucket* Family

This section introduces four novel allocation algorithms in the *X-Bucket* family that share the principle of the bucketing approach. We first give an intuition behind the bucketing approach, describe how each algorithm in the family operates within this framework, explain a few heuristics to integrate algorithms into WorkQueue and TaskVine, and detail how separate resource predictions for each resource type are combined into one final schedulable resource prediction. Furthermore, we will show that these algorithms meet all design goals as stated in Section II-D.

A. The Bucketing Approach

Figure 3a demonstrates how the bucketing approach fits inside the internal allocation and scheduling logic of a workflow system. (1) The workflow manager sends a ready task *T* to the task allocator for a resource allocation. (2) The task allocator requests a resource allocation prediction from the *Bucket Composer* for *T*. (3) The *Bucket Composer* maintains a separate state for each resource type, queries each state for a value, composes a schedulable resource allocation *A*, and sends it back to the task allocator. (4) The task allocator passes *A* to the task scheduler, which sends *T* with allocation *A* to a worker for execution. (5) Once *T* completes, the worker returns the result and resource record R and (6)(7) the task allocator forwards R to both the *Bucket Composer* and the workflow manager to update their respective states. Note how the bucketing approach centers around a *Bucket Composer* that interacts with the task allocator to perform only two operations: respond to an allocation request upon a ready task and update its internal states upon a completed task. As algorithms in the *X-Bucket* family only diverge on how to update the internal bucketing states and share the resource prediction approach, we will now explain the prediction approach and delay state updates to later subsections.

Since each resource type is tracked and updated independently, we will simplify the explanation of the prediction approach by focusing on only one resource. Figure 3b demonstrates how a task's allocation is derived via the bucketing approach given a certain bucketing state as an example. Assume that we have a running workflow with 2,000 completed tasks, each of which's memory consumption in GBs follows the normal distribution $\mathcal{N}(8, 2)$. Further assume that task 2,001 is now submitted to the *Bucket Composer*, requesting for a memory allocation prediction. Since the first 2,000 tasks already complete their executions, the *Bucket Composer* has access to these tasks' resource consumption reports. It then sorts these reports by resource value and tries to find potential clusters representing groups of tasks consuming similar amounts of memory. For now assume that the *Bucket Composer* already divided the completed tasks into 3 clusters: $(0, v_1]$, $(v_1, v_2]$, and $(v_2, v_{max}]$. 3 buckets are then constructed from this cluster discovery using two break points of v_1 and v_2, and task 2,001 is allocated with one of these buckets depending on the exact algorithm's bucket selection policy.

We now define precisely what a bucket is. Each bucket is defined by two elements: the representative value and the probability value. Let **B** be the set of all constructed buckets, and B_i be the i^{th} bucket, then the representative value of B_i is the maximum value of all reports in a bucket:

$$B_i.rep = \max_{r \in B_i.reports} (r.value),$$

and the probability value of B_i is the ratio of the number of reports contained in B_i:

$$B_i.prob = \frac{\#reports \in B_i}{\Sigma_{B_j \in \mathbf{B}} \#reports \in B_j}$$

When a new task needs to be allocated, depending on the exact bucketing algorithm, the *Bucket Composer* will choose

Algorithm 1 Greedy Bucketing

```
procedure GREEDYBUCKETING(lo, hi, L, b_type)
    if lo == hi then return [lo]
    min_cost, break_idx = ∞, None
    for i = lo to hi do
        cost = compute_greedy_cost(lo, i, hi, L, b_type)
        if cost < min_cost then
            min_cost, break_idx = cost, i
    if break_idx == hi then return [hi]
    lo_indices = GreedyBucketing(lo, break_idx, L, b_type)
    hi_indices = GreedyBucketing(break_idx+1, hi, L, b_type)
    return lo_indices.concat(hi_indices)
end procedure
```

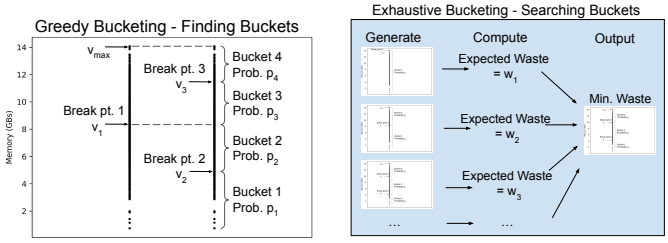
a bucket among the pre-calculated bucketing state (i.e., the list of buckets), based on the algorithm's policy and return its representative value as the prediction. In the event that the new task exceeds this allocation prediction during its execution and returns to the *Bucket Composer* for a new prediction, the *Bucket Composer* only considers buckets having their representative values greater than that of the task's last consumption value. If no bucket is available (meaning this task consumes more than any completed task), the *Bucket Composer* doubles the task's previous consumption until the task succeeds.

The bucketing approach thus satisfies three design goals, *general-purpose*, *prior-free*, *online*, as stated in Section II-D. To partly address the *robust* design goal, we notice that when workflows transition from one phase to another and thus may exhibit a large jump in resource consumption, more recent task reports serve as a better guidance to allocate subsequent tasks. To encode this guidance in the algorithms, we add a *significance* value to each task report, and the higher the value the more recent or "significant" this task report is (we discuss how to set this value in Section V). The probability value of each bucket B_i is then updated to:

$$B_i.prob = \frac{\Sigma_{r \in B_i.reports} r.sig}{\Sigma_{B_j \in \mathbf{B}} \Sigma_{r \in B_j.reports} r.sig}$$

where $r.sig$ is the significance value of report r.

The arbitrary moving resource consumption distribution aspect of the *robust* design goal is particularly challenging. To tackle this problem while following the bucketing approach, we divide it into two main sub-problems: how to optimally divide a list of completed resource reports into a bucketing state, and how to use this state in the prediction process. As we will soon see, the way the bucketing state is used for prediction affects the derivation of the bucketing state, and vice versa. For clarity, we first group *PG-Bucket* and *IG-Bucket* into a new *G-Bucket* (*Greedy Bucketing*) sub-family, and *PE-Bucket* and *IE-Bucket* into a new *E-Bucket* (*Exhaustive Bucketing*) sub-family. We then provide a high-level overview of the sub-families, which concerns the problem of dividing a list of resource reports into a bucketing state, before digging deeper into each algorithm's way of optimally finding the bucketing state and using them for prediction.



(a) Greedy strategy (b) Exhaustive strategy

Fig. 4. Different strategies in finding the optimal set of buckets between Greedy and Exhaustive algorithms.

B. The G-Bucket and E-Bucket Sub-families

1) G-Bucket: The goal of the *G-Bucket* sub-family is to find a bucketing state with the minimum amount of resource waste in a greedy, recursive manner. In each recursive step, a *G-Bucket* algorithm asks: given a list of completed resource reports, should it break that list into exactly two sublists or not, and if yes, where exactly is the break point? If the answer is yes, and assuming the break point is v_1, the algorithm breaks the list from one interval $(0, v_{max}]$ into two, $(0, v_1]$ and $(v_1, v_{max}]$, forming two new buckets of B_{v_1} and $B_{v_{max}}$. If the answer is instead no, meaning it is optimal to not break the current list into 2 buckets, then it simply returns the entire list as one bucket.

This is the heart of *G-Bucket* algorithms as shown in Algorithm 1. Since a *G-Bucket* algorithm does not know where v_1 lies, it scans through the entire list of reports and computes the expected resource waste at each point. It then chooses the break point yielding the lowest expected waste, and recursively calls itself on the two newly found buckets before joining the recursive results together into one bucketing state. However, if the break point is v_{max}, it stops the computation and returns $[v_{max}]$ as the best list of break points, or equivalently the best list of buckets. Figure 4a shows an example run of a *G-Bucket* algorithm. It begins by choosing v_1 as the break point in the first recursive step. v_1 then breaks the list of resource reports into two buckets, each of which is recursively called again to yield the final bucketing state of $[v_1, v_2, v_3, v_{max}]$ with 4 buckets, guaranteeing that each recursive step yields the minimum expected local resource waste. The only difference between *IG-Bucket* and *PG-Bucket* is in the way the bucketing state is used for prediction, and more importantly, the *compute_greedy_cost* function that computes the expected waste, as denoted by the parameter b_type.

2) E-Bucket: Instead of greedily finding the bucketing state, *E-Bucket* algorithms (see Algorithm 2) consider all possible configurations of a bucketing state in a list of reports, compute the expected resource waste associated with each configuration, and return the configuration with the minimum waste as the best bucketing state. Specifically, let L be a list of task resource reports. Since each report can be a bucket, there can be at least 1 bucket and at most $L.length$ buckets. The outer for loop thus runs from 0 to $L.length - 1$ as the number of possible break points is at least 0 and at most $L.length - 1$. The inner loop then generates all possible combinations of break

Algorithm 2 Exhaustive Bucketing

```
procedure EXHAUSTIVEBUCKETING(L, b_type)
    min_cost, break_indices = ∞, None
    for k = 0 to L.length-1 do
        for P in combinations(k, L) do
            cost = compute_exhaust_cost(P, L, b_type)
            if cost < min_cost then
                min_cost, break_indices = cost, P
    return break_indices
end procedure
```

points of length k that can be drawn from list L and returns the one with the minimum expected resource waste. Figure 4b depicts a possible execution of an *E-Bucket* algorithm. Similar to algorithms in the *G-Bucket* sub-family, *PE-Bucket* and *IE-Bucket* are also different in the way the bucketing state is used and the exact derivation of the bucketing state via the *compute_exhaust_cost* function.

C. Algorithms in Detail

We have covered the bucketing prediction approach as well as the high-level approaches (*G-Bucket*, *E-Bucket*) on finding the optimal bucketing state. We will now cover the last piece of the puzzle concerning the bucket selection policy and the exact details on the low-level modeling to calculate the expected waste on all 4 algorithms. Recall in Section IV-A that when a new task needs an allocation prediction, the *Bucket Composer* will choose a bucket depending on the algorithm's policy. Besides the approach to find the optimal bucketing state, algorithms in the *X-Bucket* family can also be divided by their bucket selection policy, namely *Incremental* and *Probabilistic*, resulting in the total of 4 algorithms.

When choosing a bucket in a bucketing state to allocate the next task, *Incremental* bucketing algorithms always ignore buckets' probability values and choose the lowest bucket possible. That is, if a task has never run before, it will always allocate using the representative value of B_1. If a task has run before and exhausted its previous resource allocation, it will always choose the bucket with the smallest representative value that is greater than the task's previous resource consumption. This approach thus provides a naturally incremental determination of allocation, but risks incurring unnecessary waste from repeated failed allocations if the new task's resource consumption lies in the higher buckets.

On the other hand, *Probabilistic* algorithms take buckets' probability values into account and select a bucket based on this distribution. This approach can improve the allocation efficiency by allowing the allocation value to skip lower buckets and focus on buckets with higher concentrations of completed tasks. This helps the allocation prediction to align better with the resource consumption distribution of tasks in the current phase. However, it risks *Failed Allocation* waste if denser buckets have lower representative values and the next task consumes a lot of resources, and *Internal Fragmentation* waste if denser buckets have higher representative values and the next task consumes minimal amounts of resources. We now detail the exact formula to compute the expected resource waste of each algorithm.

1) PG-Bucket: Assume that the *PG-Bucket* algorithm is evaluating the expected resource waste of the candidate break point v_1, or equivalently the bucketing state consisting of 2 buckets: $(0, v_1]$ and $(v_1, v_{max}]$. As this algorithm is *Probabilistic*, it will choose the allocation for the next task T based on $B_{v_1}.prob$ and $B_{v_{max}}.prob$, where $B_{v_1}.prob + B_{v_{max}}.prob = 1$. Assume that T follows the resource consumption behaviors of completed tasks, then T's consumption has a probability of $B_{v_1}.prob$ to be in $(0, B_{v_1.rep}]$ and a probability of $B_{v_{max}}.prob$ to be in $(B_{v_1.rep}, B_{v_{max}.rep}]$. Denoting the true resource consumption of T as v_{lo} if it is in B_{v_1}, and as v_{hi} if it is in $B_{v_{max}}$, we have four possible cases to consider:

- *Low consumption, low prediction*: both T's consumption and *PG-Bucket*'s prediction fall in B_{v_1}. This event happens with a probability of $(B_{v_1}.prob)^2$ and incurs an expected resource waste of $W_{lo,lo} = B_{v_1}.prob^2(B_{v_1}.rep - v_{lo})$.
- *Low consumption, high prediction*: this event happens with a probability of $B_{v_1}.prob \cdot B_{v_{max}}.prob$ and incurs an expected resource waste of $W_{lo,hi} = B_{v_1}.prob \cdot B_{v_{max}}.prob(B_{v_{max}}.rep - v_{lo})$.
- *High consumption, low prediction*: this event causes T to exhaust the first allocation and require a bigger allocation. It happens with a probability of $B_{v_{max}}.prob \cdot B_{v_1}.prob$ and incurs an expected resource waste of $W_{hi,lo} = B_{v_{max}}.prob \cdot B_{v_1}.prob(B_{v_1}.rep + B_{v_{max}}.rep - v_{hi})$.
- *High consumption, high prediction*: both T's consumption and *PG-Bucket*'s prediction fall in $B_{v_{max}}$. This event happens with a probability of $B_{v_{max}}.prob^2$ and incurs an expected resource waste of $W_{hi,hi} = B_{v_{max}}.prob^2 \cdot (B_{v_{max}}.rep - v_{hi})$.

The total expected resource waste for *PG-Bucket* is then $W = W_{lo,lo} + W_{lo,hi} + W_{hi,lo} + W_{hi,hi}$.

Since v_{lo} and V_{hi} are not known in advance as stated in Section II-B, they are estimated using a weighted average of resource reports in each bucket that they fall in:

$$v_{lo} = \frac{\Sigma_{r \in B_{v_1}.reports} r.value \cdot r.sig}{\Sigma_{r \in B_{v_1}.reports} r.sig}$$

$$v_{hi} = \frac{\Sigma_{r \in B_{v_{max}}.reports} r.value \cdot r.sig}{\Sigma_{r \in B_{v_{max}}.reports} r.sig}$$

2) IG-Bucket: We will use the same setup as that of *PG-Bucket* to describe the *IG-Bucket* algorithm. However, since *IG-Bucket* is *Incremental*, it will always allocate T using the B_{v_1} bucket in the first try, and fall back on $B_{v_{max}}$ if T exhausts the first allocation. We thus have two cases to consider:

- *Low consumption*: T's consumption falls in B_{v_1}, and the expected resource waste is $W_{lo} = B_{v_1}.prob(B_{v_1}.rep - v_{lo})$.
- *High consumption*: T's consumption falls in $B_{v_{max}}$, and the expected resource waste is $W_{hi} = B_{v_{max}}.prob(B_{v_1}.rep + B_{v_{max}}.rep - v_{hi})$.

The expected waste for break point v_1 is thus $W = W_{lo} + W_{hi}$.

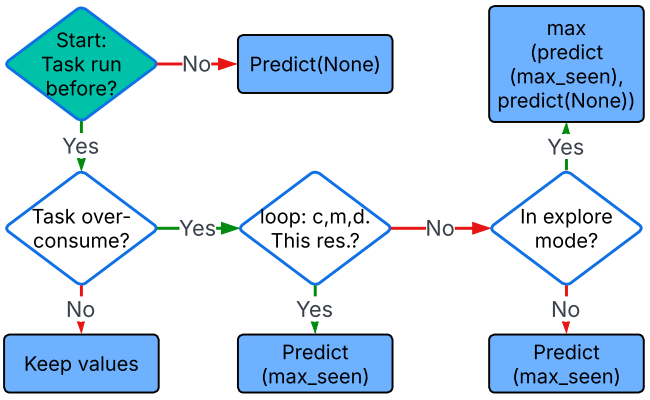


Fig. 5. Flowchart of *Bucket Composer*'s prediction decisions. A brief explanation of terms follows. *Predict(None)*: predict a new allocation for a new task. *Keep values*: do not change the current allocation values. *Predict(max_seen)*: predict a new allocation based on the task's maximum consumption seen so far.

3) PE-Bucket: Instead of scanning the list of resource reports to find a break point, *PE-Bucket* inputs a candidate list of buckets **B** of length N and compute its expected resource waste. Assume that the next task follows the resource consumption distribution of completed tasks and consumes v_i within bucket B_i. v_i is then estimated the same way *G-Bucket* algorithms estimate v_{lo} and v_{hi}:

$$v_i = \frac{\Sigma_{r \in B_i.reports} r.value \cdot r.sig}{\Sigma_{r \in B_i.reports} r.sig}$$

Since *PE-Bucket* is *Probabilistic*, the next task's consumption and allocation prediction can be anywhere in N buckets, resulting in N^2 cases to handle. Let $T[i,j]$ be the expected resource waste when the next task's consumption and allocation fall within buckets B_i and B_j, respectively. If $i \leq j$, then

$$T[i,j] = B_j.rep - v_i,$$

as the allocation from B_j is sufficient for the next task's execution. Otherwise,

$$T[i,j] = B_j.rep + \Sigma_{k=j+1}^{N} \frac{B_k.prob}{\Sigma_{m=j+1}^{N} B_m.prob} \cdot T[i,k]$$

Since $i > j$, T consumes more than its allocation and thus forces a retry with higher buckets in $[j+1, N]$. The term $\frac{B_k.prob}{\Sigma_{m=j+1}^{N} B_m.prob}$ renormalizes the probabilities to only the higher buckets, and $T[i,k]$ is defined similarly to $T[i,j]$. Intuitively, $T[i,j]$ represents the sum of the current resource waste from *Failed Allocation* and the expected resource waste upon the next allocation. Note that if $j < i$ and $j < k$, then $T[i,j]$ depends on $T[i,k]$. The table T then should be filled from the last column to the first column by row. After computing all entries in T, the expected resource waste of the bucketing state **B** is then:

$$W_{\mathbf{B}} = \Sigma_{i=1}^{N} \Sigma_{j=1}^{N} B_i.prob \cdot B_j.prob \cdot T[i,j],$$

as the probability that the next task falls within B_i and *PE-Bucket* chooses B_j is $B_i.prob * B_j.prob$.

4) IE-Bucket: We will also use the same setup of *PE-Bucket* to describe the *IE-Bucket* algorithm. Since *IE-Bucket* is *Incremental*, we only have N cases to handle as only the next task's consumption can vary between N buckets. Let the next task's consumption be v_i falling in the B_i bucket. The total allocation of this task is then $\Sigma_{j=1}^{i} B_j.rep$, as it must be allocated by all lower buckets and the true one. The probability that the next task's consumption failling in B_i is $B_i.prob$. Therefore, for a given bucketing state **B**, its expected waste under the *IE-Bucket*'s policy is:

$$W_{\mathbf{B}} = \Sigma_{i=1}^{N} B_i.prob(\Sigma_{j=1}^{i} B_j.rep - v_i)$$

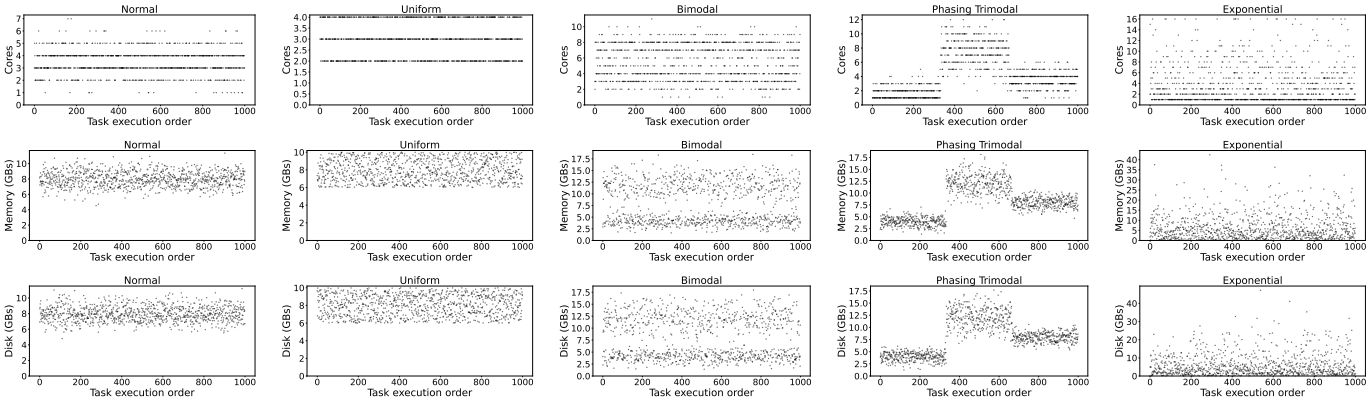
D. Analysis and Optimization of X-Bucket Algorithms' Complexity

We now analyze the time complexity of all four *X-Bucket* algorithms assuming the size of the list L in Algorithms 1 and 2 is n. For *G-Bucket* algorithms, the for loop in Algorithm 1 is $O(n)$ on its own as each recursive iteration must scan all points between lo and hi. In each loop iteration, both *PG-Bucket* and *IG-Bucket* must call their own computations in the $compute\_greedy\_cost$ function, which has a time complexity of $O(n)$ as it needs to accumulate the buckets' probability values from all points and compute the estimated v_{lo} and v_{hi} values. Despite the two recursive calls on smaller input sizes at the end, our experience running *G-Bucket* algorithms shows that the total number of recursions is bounded by a constant (more details in V-A). Therefore, *G-Bucket* algorithms have a total time complexity of $O(n^2)$.

For *E-Bucket* algorithms, the two loops in Algorithm 2 have a total time complexity of $O(n \cdot 2^n)$ as they jointly generate the powerset of list L. The complexity of the $compute\_exhaust\_cost$ function for both *PE-Bucket* and *IE-Bucket* is $O(n^3)$ as the T table has up to n^2 entries, and each entry can take $O(n)$ time due to the renormalization process. *E-Bucket* algorithms thus have a massive total time complexity of $O(n^4 \cdot 2^n)$. To address this exponential time complexity, we first modify the call to $combinations(k, L)$ to only generate one subset that divides all points most evenly into k buckets as follows:

1) we first form a list of $k-1$ candidate break points L to break the list of points into regions of equal sizes, so $L[i] = \frac{v_{max} \cdot i}{k}$ for i in $[1, k-1]$.
2) for each candidate break point, we then map its value to the closest point that has a lower value than it and remove all duplicate or empty mappings.
3) we then return newly found points as the actual list of break points to be considered.

To further cut down the unnecessary searches with an excessive number of break points, we limit the number of iterations in the outer loop to a constant chosen empirically as discussed later in Section V-A. This optimization then collapses the time complexity of *E-Bucket* algorithms down to $O(n)$, which is needed to compute buckets' probability values and estimate v_i values.



Normal	Uniform	Bimodal	Phasing Trimodal	Exponential
Cores: $N(4,1)$	$U(2,4)$	$N_1(4,1)N_2(8,1)$	$N_1(2,1)N_2(8,2)N_3(4,1)$	$E(4)$
Memory: $N(8,1)$	$U(6,10)$	$N_1(4,1)N_2(12,2)$	$N_1(4,1)N_2(12,2)N_3(8,1)$	$E(6)$
Disk: $N(8,1)$	$U(6,10)$	$N_1(4,1)N_2(12,2)$	$N_1(4,1)N_2(12,2)N_3(8,1)$	$E(6)$

Fig. 6. Resource consumption of tasks in five synthetic workflows. Resource types vary from cores, memory, to disk by row, from top to bottom. In each plot, the x-axis shows the task ID, numbered from 1 to 1000, and the y-axis shows the magnitude of resource consumption.

E. Implementation of the X-Bucket Family

We detail several refinements to tie the remaining loose knots between the *Bucket Composer* and other components of a dynamic workflow system. All algorithms in the *X-Bucket* family expect a list of resource reports from completed tasks. To get the very first reports, these algorithms run in an exploratory mode for a while, allocating tasks a predefined amount of resources with a default backup strategy, until they collect a sufficient number of reports.

As shown in Figure 3a, the *Bucket Composer* maintains a separate bucketing state for each resource type, each of which can be configured to run any of the 4 *X-Bucket* algorithms. The pseudocode for the *Bucket Composer* is as follows:

```
class BucketComposer:
    def __init__(self, params):
        #initialize the list of bucketing states,
        #one per resource type

    def add(self, task_report):
        #add task report to the appropriate
        #bucketing state
        #also call the appropriate bucketing state
        #to re-compute the list of buckets

    def predict(self, task):
        #get an allocation prediction from each state
        #and join them into one schedulable allocation
```

In practice, not all allocation predictions of a task result in a scheduling and execution with such allocation. Many reasons arise: the workflow system may try to determine the resource demands of remaining tasks and ask for their predicted allocations, or a task-to-node scheduling requires an allocation in the first step, but fails in the later steps, requiring another allocation prediction, etc. Rerunning a given bucketing algorithm every time an allocation is needed is unnecessary and expensive. To avoid this, Figure 5 shows a flowchart of decisions *Bucket Composer* takes before allocating a given task. We will skip the first two decisions as they are self-explanatory, and focus on the latter two. For a task *T*, upon its over-consumption, *Bucket Composer* detects the resource types that were over-consumed and generates a new prediction based on their last consumption values. For resource types that are not over-consumed, *Bucket Composer* projects the last consumption value to its bucket's representative value via *predict(max_seen)* to stabilize the allocation. If a bucketing state is in the exploratory mode however, *Bucket Composer* also respects its default value and takes the max of these two values for allocation.

V. Evaluation

In this section, we first describe all experimental settings used to evaluate the *X-Bucket* family. In addition to three production workflows, we generate five synthetic workflows to understand and evaluate the performance and robustness of the algorithms. We then present our analyses of results via *AWE*, *ATE*, and *Resource Waste* metrics. We conclude this section by empirically showing that *PE-Bucket* is the best algorithm among the *X-Bucket* family.

A. Settings

We implement the algorithms in the task allocator component of WorkQueue and TaskVine to minimize changes to production workflows. Each experiment is run on opportunistic workers with 16 cores and 64 GBs of memory and disk, and the number of workers vary between 20 to 50 depending on the local HTCondor cluster's availability. In all experiments, we set the significance value of a task report to the task ID, so task with ID 1 has its resource report of significance value of 1, and so on. For all *X-Bucket* algorithms, we notice from our experience during experiment runs that the number of buckets rarely exceeds 10 at any given time, so we limit k to 10 in the outer for loop of Algorithm 2. To collect the very first resource reports, all algorithms are initially run in the exploratory mode and allocate each task 1 core and 1 GB of memory and disk until 10 reports are collected. Similar to

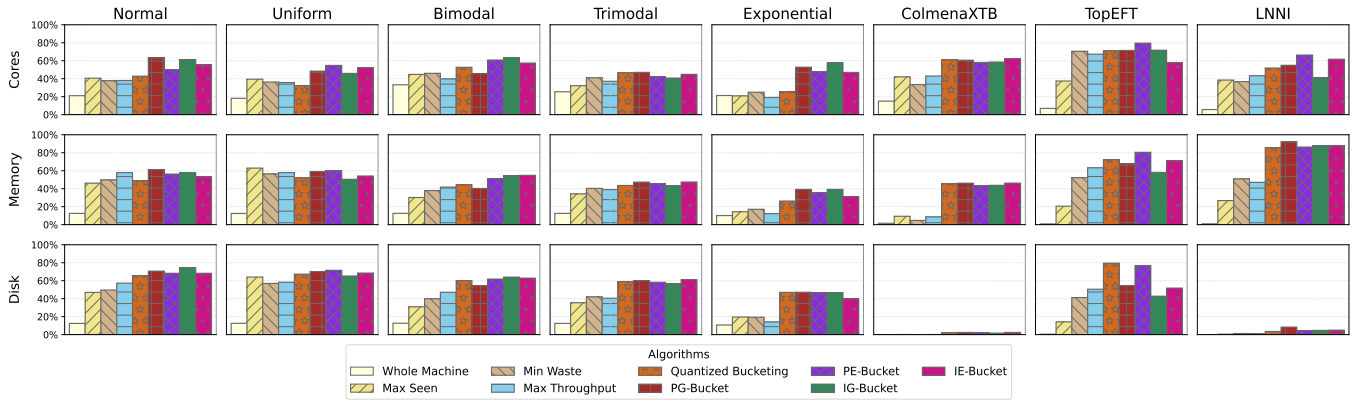


Fig. 7. Absolute Workflow Efficiency in cores, memory, and disk of 8 workflows across 9 allocation algorithms.

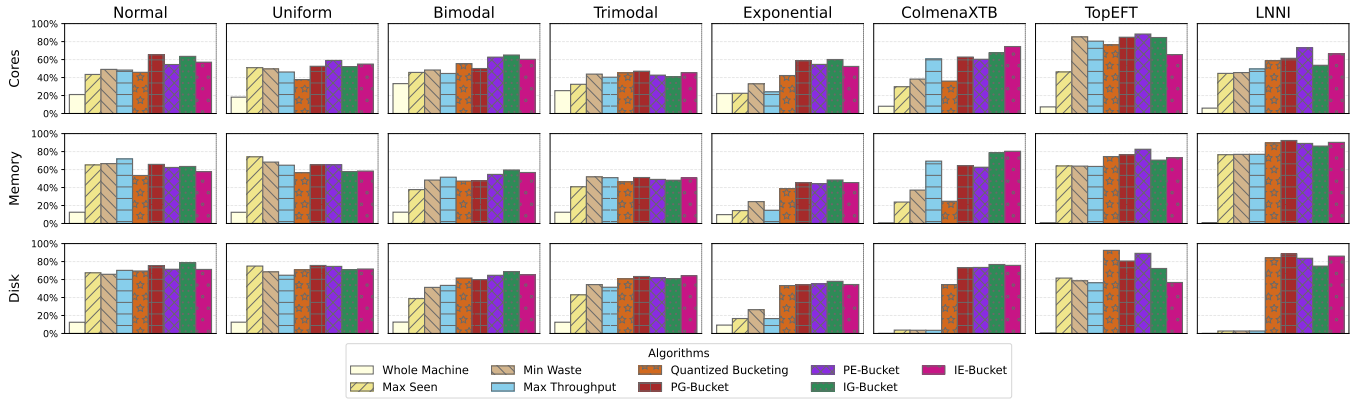


Fig. 8. Average Task Efficiency in cores, memory, and disk of 8 workflows across 9 allocation algorithms.

the exponential backup described in Section IV-A, if a task exhausts any type of resources during this mode, it is retried by doubling the previous amount of resource consumption.

Five synthetic workflows, *Normal, Uniform, Bimodal, Phasing Trimodal,* and *Exponential*, are generated to evaluate the robustness of the *X-Bucket* algorithms via a diverse set of resource consumption distributions, each of which contains 1,000 tasks. Figure 6 shows the distribution parameters for each resource type that were used to generate the workflows. Each synthetic workflow represents a common behavior in production workflows: *Normal* and *Uniform* for inherent stochasticity, *Bimodal* for task specialization, *Phasing Trimodal* for the arbitrary moving resource consumption distributions between phases, and *Exponential* for outliers.

To compare the performance of our algorithms, we use two naive algorithms, *Whole Machine* and *Max Seen*, and three alternative algorithms that align to our design goals, namely *Min Waste*, *Max Throughput*, and *Quantized Bucketing*. Naive algorithms represent a reasonable lower bound of efficiency and are straightforward: *Whole Machine* simply allocates each task the total resource capacity of a worker, while *Max Seen* allocates each task the maximum resource value seen as the workflow runs. *Min Waste* and *Max Throughput* follow the descriptions of respective algorithms in [12] where they both provide a small allocation per task for the first try and fall back to the largest allocation upon failure, yet differ in the formulations of the exact allocations according to their optimization criteria (i.e., minimizing resource waste or maximizing tasks' throughput). Finally, *Quantized Bucketing* (not in the *X-Bucket* family) follows its description in [8], in which it forms buckets of resources based on their densities using a preset hyperparameter.

B. Resource Efficiency and Resource Waste Analysis

Figure 7 shows the *Absolute Workflow Efficiency* of all workflows under 9 algorithms. For synthetic workflows, a quick glance shows that workflows with "harder" distributions, i.e., more stochasticity factors like task specialization effects or phasing behaviors, tend to have a lower resource efficiency (especially in memory and disk) due to the increase in intrinsic risks of resource waste associated with more complex workflows. Inspecting individual algorithms' performance, we can see that *Whole Machine* always performs poorly as expected. *Max Seen* represents a significant resource efficiency jump from the previous baseline as it is no longer static: resource allocation predictions dynamically change to match the maximum resource value it has seen so far. The group of *Min Waste, Max Throughput,* and *Quantized Bucketing* yields a similar range of resource efficiency across synthetic workflows, but is usually 5-20% less efficient than the *X-Bucket* algorithms

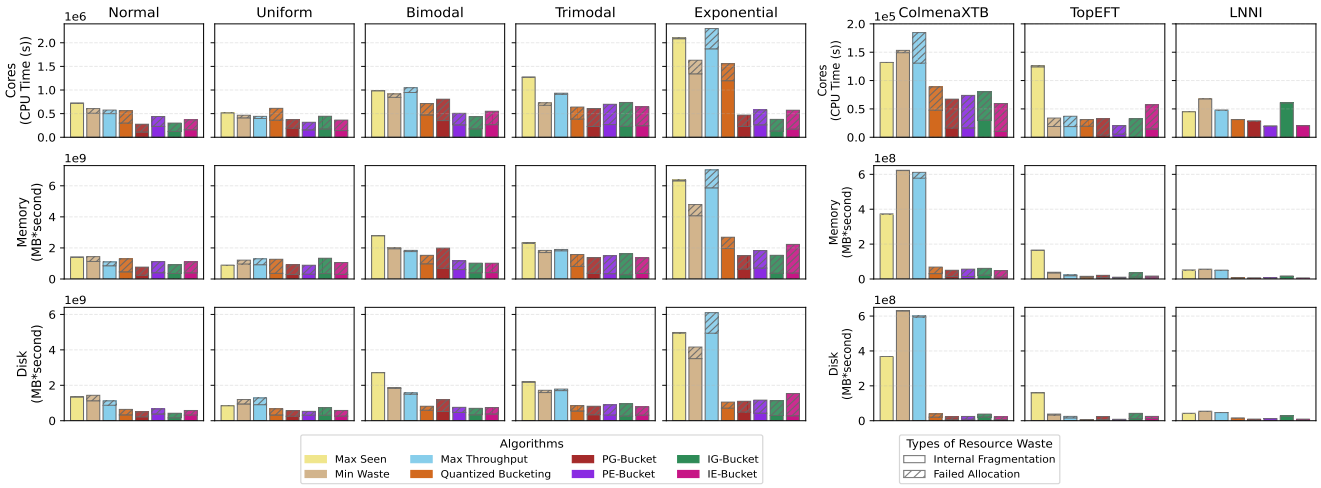


Fig. 9. Resource Waste in cores, memory, and disk of 8 workflows across 8 allocation algorithms.

(except the *Exponential* workflow, with up to 40% difference in core efficiency). The *X-Bucket* algorithms perform well in general, either comparable to or significantly more efficient than alternative algorithms on "easier" workflows like *Normal* and *Uniform*, with resource efficiency hovering around 60-80%. In more stochastic workflows like *Bimodal* and *Trimodal*, the *X-Bucket* algorithms consistently show a large difference in resource efficiency compared to alternative algorithms. This is due to *X-Bucket* algorithms following design goals in Section II-D that allows them to better absorb the inherent randomness from these workflows and derive better resource allocation predictions. The benefits from following the design goals are more apparent in the *Exponential* workflow: only the *X-Bucket* algorithms can manage to derive allocation predictions that are resistant to the infrequent outliers while alternative algorithms cannot and thus perform much poorer.

In production workflows, observe that the disk efficiency is almost 0 in *ColmenaXTB* and *LNNI* workflows due to a combination of low disk consumption of tasks ($<$ 20 MBs, see Figure 2) and large exploration cost to collect initial resource reports, either with the *X-Bucket* algorithms and *Quantized Bucketing* allocating 1 GB of disk or with other algorithms allocating 64 GBs of disk. We will thus focus more on the core and memory efficiency. In the ColmenaXTB workflow, we see a familiar trend in resource efficiency between algorithms as the efficiency gradually gets better from the *Whole Machine* baseline to *Quantized Bucketing*. The group of *Max Seen, Min Waste,* and *Max Throughput* is not able to timely detect the phase change between *evaluate_mpnn* and *compute_atomization_energy* and effectively adjust the allocation predictions, resulting in weaker resource efficiency. On the other hand, *X-Bucket* algorithms are able to capture the phase change and properly adjust allocation predictions, leading to much better resource efficiency. The *TopEFT* workflow is mostly dominated by *processing* tasks (87.4% of all tasks), resulting in similar resource efficiency between algorithms. Finally, note that the *LNNI* workflow has a *Normal*-like core consumption and mostly homogeneous memory consumption, leading to a relatively weaker core efficiency and stronger memory efficiency. Figure 8 shows the *Average Task Efficiency* of all workflows under 9 algorithms. Note that this metric tracks the quality of resource allocation predictions over a long run and lessens the impact of the exploration cost. We can see that the resource efficiency in almost all cases increases as the exploration cost's effect is reduced by the averaging of individual tasks' resource efficiency. However, the general shapes of individual workflow-resource plots remain the same, with the *X-Bucket* algorithms consistently outperform other algorithms in resource efficiency in both metric types.

Figure 9 shows the resource waste of all workflows across 8 allocation algorithms in 3 resource types broken down into *Internal Fragmentation* and *Failed Allocation* (we skip *Whole Machine* for better visualization). We can now observe the algorithm tendencies on whether to risk *Internal Fragmentation* or *Failed Allocation* to derive an allocation prediction. *Max Seen* almost always incurs *Internal Fragmentation* waste, which is in accordance with its strategy of allocating the maximum resource seen. *Min Waste* and *Max Throughput* are not as conservative and have slightly more *Failed Allocation* waste as they attempt to allocate small allocations first before committing larger allocations. *Quantized Bucketing* moves even closer to *Failed Allocation* as its exploration strategy is to allocate tasks with a small value first, then exponentially increase them later. The *X-Bucket* algorithms derive the most aggressive allocation predictions, especially *Incremental* algorithms, as they always try a small resource allocation first and dislike risking a large *Internal Fragmentation* waste.

C. Quantifying the Best X-Bucket Algorithm

We now attempt to quantify the best allocation algorithm via deep analyses into above results. Table I shows the *H-CRE* and P-CRE metrics of all algorithms across 8 workflows. The last column, *Aggregate*, compiles the final resource- and workflow-agnostic efficiency by taking the geometric mean of all resource-agnostic efficiency for each workflow. For example, note that *PG-Bucket* and *PE-Bucket* lead the *P-CRE*

TABLE I
CRE METRICS OF 9 ALGORITHMS ACROSS 8 WORKFLOWS (IN %).

	Normal		Uniform		Bimodal		Trimodal		Exponential		Colmena		TopEFT		LNNI		Aggregate	
Algorithms	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE	H-CRE	P-CRE
Whole Machine	14.9	17.6	14.2	15.9	17.4	23.7	15.9	19.9	13.2	16.4	0.8	5.5	1.4	3.2	0.4	2.4	4.9	9.9
Max Seen	44.5	42.5	54.2	46.5	34.7	39.1	33.9	33.0	18.0	18.6	3.8	20.1	22.2	29.8	8.1	27.8	20.8	30.7
Min Waste	45.3	41.6	48.9	42.4	41.1	43.1	41.1	40.9	20.2	22.0	2.1	13.6	53.3	62.7	13.3	34.2	24.1	34.5
Max Throughput	50.1	44.0	49.4	42.3	42.8	40.7	38.9	37.9	14.9	16.5	3.7	19.9	59.9	65.1	13.1	37.0	25.5	35.0
Quantized Bucketing	51.6	45.5	48.3	38.6	51.9	50.3	49.2	46.2	31.6	26.5	17.5	46.9	74.1	71.7	25.0	52.7	40.0	45.7
PG-Bucket	**64.8**	**62.8**	58.5	52.4	46.3	44.1	**51.1**	**47.7**	46.0	48.0	18.9	47.5	64.1	69.3	**34.9**	58.4	**45.2**	53.2
PE-Bucket	57.6	52.5	**61.6**	**57.0**	57.7	57.7	48.2	43.9	42.9	43.6	17.5	45.0	**78.8**	**79.7**	28.6	**62.5**	45.0	**54.2**
IG-Bucket	63.9	60.6	53.2	48.0	**60.5**	**60.6**	46.3	42.2	**47.3**	**50.8**	16.3	44.9	56.2	65.6	25.7	46.4	42.5	51.8
IE-Bucket	58.6	55.3	57.9	53.5	58.3	56.9	50.7	46.3	38.7	41.1	**19.2**	**48.5**	59.6	61.0	30.0	60.5	43.7	52.5

TABLE II
AVERAGE TIME (IN μS) TO COMPUTE A NEW BUCKETING STATE AND A NEW ALLOCATION PREDICTION. (COLUMNS) THE SIZE OF THE LIST OF COLLECTED RESOURCE REPORTS. (ROWS) THE *X-Bucket* ALGORITHMS.

	List Size				
Algorithm	10	200	1000	2000	5000
PG-Bucket	15	1,490	38,585	157,239	1,097,141
PE-Bucket	17	199	900	1,396	3,863
IG-Bucket	12	1,506	39,237	161,410	1,130,565
IE-Bucket	16	181	863	1,360	3,792

aggregate metric with power-denominated efficiency of 53.2% and 54.2% respectively. An approximate interpretation is for every watt of resource allocation, when these algorithms are run on the 8 workflows above, we expect that 0.532 or 0.542 watts are efficiently used. Overall, we can see that the *X-Bucket* algorithms almost always outperform all other algorithms, with the algorithm yielding the best efficiency exclusively in the *X-Bucket* family. When aggregating individual algorithm's CRE metrics on all workflows (last 2 columns), we further observe the clustering of algorithms based on their prediction strategy. *PG-Bucket* and *PE-Bucket* lead in both *H-CRE* and *P-CRE* metrics due to their *Probabilistic* nature, allowing them to adapt quicker to the current running resource consumption distribution of a given workflow. *IG-Bucket* and *IE-Bucket* follow behind only slightly (about 2-3% reduction) as they share the bucketing approach and provide a relatively more stable series of predictions per task - probabilistic values of buckets are not required hence not used - but incur more waste from the *Incremental* predictions as tasks must always be allocated from the lowest bucket to the highest. Furthermore, these results show the predictive value of the general bucketing framework as replacing one bucket selection policy with another (i.e., probabilistic versus incremental) only yields minor efficiency reductions and all X-Bucket algorithms still substantially outperform alternative ones. *Quantized Bucketing* balances the conservative and aggressive predictions, yielding a more moderate mix of *Internal Fragmentation* and *Failed Allocation* waste, and thus trails behind the *X-Bucket* algorithms by about 5-9%. *Min Waste* and *Max Throughput* always try one small allocation before falling back to the maximum allocation, thus are more vulnerable to *Internal Fragmentation* and shows a significant gap in efficiency, between 16 - 20% compared to the *X-Bucket* algorithms. Finally, *Max Seen* and *Whole Machine* have the worst efficiency as expected due to their simple nature.

Table II shows the average time it takes to compute a new bucketing state and a new allocation prediction for all *X-Bucket* algorithms with increasing numbers of collected resource reports. We can see that the *G-Bucket* algorithms are quite expensive with a $O(n^2)$ time complexity, taking around 1 second on average to derive a new allocation prediction due to their recursive nature. On the other hand, *E-Bucket* algorithms with their optimized computations as described in Section IV-E show a linear growth in time complexity, only taking around 3.9 ms to update their bucketing states and derive a schedulable resource allocation. Note that each task does not need to trigger a full re-computation of the bucketing state. Specifically, a sequence of ready tasks can share the same bucketing state if there's no completed tasks to update the bucketing state, and a sequence of completed tasks can be batched into one big bucketing state update if no ready tasks are available.

In conclusion, results from Tables I and II show that *PE-Bucket* is empirically the best algorithm in prediction quality, resource efficiency, and runtime. In the bigger picture, above results show that the bucketing approach accurately models the uncertainty and stochasticity problems as detailed in Sections II-B and II-D. By following the four design goals of *general-purpose, prior-free, online*, and *robust*, *X-Bucket* algorithms are able to find useful buckets from collected data and utilize them correctly to derive quality allocation predictions, thus show much better resource efficiency in a diverse set of workflows compared to alternative algorithms.

VI. RELATED WORK

The study of workloads' resource consumption efficiency has produced a large amount of algorithms, strategies, and heuristics, with a number of research groups tracked and compiled them over time. Pupykina et al. [26] study strategies for memory consumption management of tasks in HPC systems, which does not account for other resource types like cores and disk. Witt et al. [10] extensively survey approaches of modeling tasks' resource consumption using black-box machine learning models. We believe that these approaches are promising in certain scenarios, but require a large amount of labeled data, model retraining and fine-tuning from time to time, and thus necessitate careful considerations to work as an online algorithm.

Other works define and optimize different objectives to improve the execution of workflows. Zhang et al. [9] apply reinforcement learning techniques to optimize the average task wait time via a novel scheduling inspector module. Thekkepurayil et. al. [27] optimize the scheduling of workflows in clouds using several objectives, including optimal resource consumption, quality of services, load balancing, etc. Li et al. [28] instead draw the context of geo-distributed data centers and focus on scheduling decisions to make sure workflows are fault-tolerant and data are placed efficiently. Several groups optimize the energy consumption of workflows in clouds with a budget constraint, as presented by Choudhary et al. [29] and Taghinezhad-Niar et al. [30]. Cohen et al. [31] focus on modeling resource overcommitment in clouds for better resource utilization and lower compute cost. Medeiros et al. [32] introduce a real-time adjustment system for memory limits of Kubernetes' pods. In this paper, we model the resource allocation problem generically with only two entities: *Task* and *Allocation*, and thus believe that our solution can complement these work and be applied to other cloud and HPC systems.

Other research groups extract workflow- or task-specific information when optimizing the resource waste of workflows. Rodrigo et al. [33] focus on the turnaround time of a workflow by analyzing the worflows' task graphs. Tanash et al. [34] train several machine learning models using tasks' metadata to predict their memory consumption. Witt et al. [35] instead use the size of task inputs to infer their resource consumption. Rodrigues et al. [36] train a machine learning model using tasks' LSF specification to process and infer their memory consumption. We believe that this approach is logical in scenarios where an important or popular workload is repeatedly or periodically run with the assumption of minimal between-run differences.

Many papers also rely on a collected list of resource reports of completed tasks to predict resource allocations of subsequent tasks during a workflow run. Tovar et al. [12] introduce two algorithms that aim to minimize waste and maximize throughput, each relying on the policy of at-most-once retry. *X-Bucket* algorithms instead relax the policy of at-most-once retry by using a bounded list of buckets. Phung et al. [8] study the heterogeneity of tasks and predict task allocations based on their categories using the k-means and quantized clustering methods. Fan et al. [37] apply reinforcement learning techniques in task scheduling depending on the availability of the local cluster instead of predicting tasks' resource consumption. We believe *X-Bucket* algorithms demonstrate a hybrid approach between reinforcement learning and clustering, where the set of buckets are discovered via the latter and the former selects a bucket among them to allocate the next task. Considering the general effectiveness of the bucketing framework with minimal performance reduction between two bucket selection policies - *Probabilistic* and *Incremental* - we believe another policy based on reinforcement learning can seamlessly integrate into the bucketing framework.

VII. Conclusion

The resource allocation problem in dynamic workflows is challenging due to the uncertainty of a resource allocation prediction for a dynamic task, the internal and external stochasticity of workflow behaviors, and the large number of tasks in workflows. We argue that a candidate solution must follow four design goals, *general-purpose, prior-free, online, robust*, in order to navigate the problem's complexity and derive quality resource allocation predictions. By strictly following these design goals, we develop *X-Bucket*, a family of four bucketing algorithms that significantly outperform alternative algorithms in prediction quality and workflow resource efficiency on a set of eight diverse synthetic and production workflows. We believe that our algorithms' results serve as evidence for our correct modeling and characterization of the resource allocation problem, and their impacts are beyond our experimental testbed and not limited to any specific workflows or underlying systems. For future work, extensions of the *X-Bucket* algorithms to other resource types (e.g., GPUs, wall time) are straightforward provided that tasks require an upper limit of these resource types and can be monitored of their resource consumption with negligible overhead. Notable challenges include a careful implementation of the extension as it requires a thorough update to a task's entire lifecycle, including creation, submission, allocation, scheduling, execution, monitoring, and retrieval.

Acknowledgment

This work was supported by National Science Foundation Grant OAC-1931348. We thank Ben Tovar, Logan Ward, Kevin Lannon, and Kelci Morman for supporting us with relevant code and data of production workflows. We use Gemini 3 to improve the presentation quality of plots and tables. WorkQueue is available at https://ccl.cse.nd.edu/software/workqueue/. TaskVine is available at https://ccl.cse.nd.edu/software/taskvine/.

References

[1] Y. Babuji, A. Woodard, Z. Li, D. S. Katz, B. Clifford, R. Kumar, L. Lacinski, R. Chard, J. M. Wozniak, I. Foster *et al.*, "Parsl: Pervasive parallel programming in python," in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, 2019, pp. 25–36.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, "Ray: A distributed framework for emerging {AI} applications," in *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, 2018, pp. 561–577.

[3] M. Rocklin *et al.*, "Dask: Parallel computation with blocked algorithms and task scheduling." in *SciPy*, 2015, pp. 126–132.

[4] D. G. Smith, D. Altarawy, L. A. Burns, M. Welborn, L. N. Naden, L. Ward, S. Ellis, B. P. Pritchard, and T. D. Crawford, "The molssi qcarchive project: An open-source platform to compute, organize, and share quantum chemistry data," *Wiley Interdisciplinary Reviews: Computational Molecular Science*, vol. 11, no. 2, p. e1491, 2021.

[5] A. C. Real, G. P. Luz, J. Sousa, M. Brito, and S. Vieira, "Optimization of a photovoltaic-battery system using deep reinforcement learning and load forecasting," *Energy and AI*, vol. 16, p. 100347, 2024.

[6] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, J. Gonzalez, K. Goldberg, and I. Stoica, "Ray rllib: A composable and scalable reinforcement learning library," *arXiv preprint arXiv:1712.09381*, vol. 85, p. 245, 2017.

[7] "Software and computing for run 3 of the atlas experiment at the lhc," *The European Physical Journal C*, vol. 85, no. 3, p. 234, 2025.

[8] T. S. Phung, L. Ward, K. Chard, and D. Thain, "Not all tasks are created equal: Adaptive resource allocation for heterogeneous tasks in dynamic workflows," in *2021 IEEE Workshop on Workflows in Support of Large-Scale Science (WORKS)*. IEEE, 2021, pp. 17–24.

[9] D. Zhang, D. Dai, and B. Xie, "Schedinspector: A batch job scheduling inspector using reinforcement learning," in *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*, 2022, pp. 97–109.

[10] C. Witt, M. Bux, W. Gusew, and U. Leser, "Predictive performance modeling for distributed batch processing using black box monitoring and machine learning," *Information Systems*, vol. 82, pp. 33–52, 2019.

[11] Y. Fan, Z. Lan, T. Childers, P. Rich, W. Allcock, and M. E. Papka, "Deep reinforcement agent for scheduling in hpc," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 807–816.

[12] B. Tovar, R. F. Da Silva, G. Juve, E. Deelman, W. Allcock, D. Thain, and M. Livny, "A job sizing strategy for high-throughput scientific workflows," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 2, pp. 240–253, 2017.

[13] P. Bui, D. Rajan, B. Abdul-Wahid, J. Izaguirre, and D. Thain, "Work queue+ python: A framework for scalable scientific ensemble applications," in *Workshop on python for high performance and scientific computing at sc11*. Seattle, WA, 2011.

[14] B. Sly-Delgado, T. S. Phung, C. Thomas, D. Simonetti, A. Hennessee, B. Tovar, and D. Thain, "Taskvine: Managing in-cluster storage for high-throughput data intensive workflows," in *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 1978–1988.

[15] L. Ward, G. Sivaraman, J. G. Pauloski, Y. Babuji, R. Chard, N. Dandu, P. C. Redfern, R. S. Assary, K. Chard, L. A. Curtiss *et al.*, "Colmena: Scalable machine-learning-based steering of ensemble simulations for high performance computing," in *2021 IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC)*. IEEE, 2021, pp. 9–20.

[16] A. Basnet *et al.*, "Topeft/topcoffea: Topcoffea 0.1 (v0.1)," (2021), zenodo. Available: https://doi.org/10.5281/zenodo.5258003.

[17] T. S. Phung, C. Thomas, L. Ward, K. Chard, and D. Thain, "Accelerating function-centric applications by discovering, distributing, and retaining reusable context in workflow systems," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024, pp. 122–134.

[18] D. Thain, T. Tannenbaum, and M. Livny, "Distributed computing in practice: the condor experience," *Concurrency and computation: practice and experience*, vol. 17, no. 2-4, pp. 323–356, 2005.

[19] T. S. Phung and D. Thain, "Adaptive task-oriented resource allocation for large dynamic workflows on opportunistic resources," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 300–311.

[20] DIGITALEUROPE, "Server efficiency metric from SERT worklet results," DIGITALEUROPE, Brussels, Belgium, Tech. Rep., December 2016. [Online]. Available: https://cdn.digitaleurope.org/uploads/2019/01/DE%20PP%20on%20server%20efficiency%20metric%20from%20SERT%20worklet%20results%2020161214.pdf

[21] Standard Performance Evaluation Corporation (SPEC), "The sert 2 metric and the impact of server configuration," Standard Performance Evaluation Corporation (SPEC), Gainesville, VA, Tech. Rep., March 2021. [Online]. Available: https://www.spec.org/sert2/SERT-metric.pdf

[22] T. S. Phung, B. Clifford, K. Chard, and D. Thain, "Maximizing data utility for hpc python workflow execution," in *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 637–640.

[23] "Colmena," 2021 [Online], exaLearn and Parsl Teams. Available: https://colmena.readthedocs.io/en/latest/index.html.

[24] "Coffea," 2021 [Online], fermi National Accelerator Laboratory. Available: https://github.com/CoffeaTeam/coffea.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[26] A. Pupykina and G. Agosta, "Survey of memory management techniques for hpc and cloud computing," *IEEE Access*, vol. 7, pp. 167 351–167 373, 2019.

[27] J. Kakkottakath Valappil Thekkepurayil, D. P. Suseelan, and P. M. Keerikkattil, "Multi-objective scheduling policy for workflow applications in cloud using hybrid particle search and rescue algorithm," *Serv. Oriented Comput. Appl.*, vol. 16, no. 1, p. 45–65, mar 2022. [Online]. Available: https://doi.org/10.1007/s11761-021-00330-4

[28] C. Li, J. Liu, M. Wang, and Y. Luo, "Fault-tolerant scheduling and data placement for scientific workflow processing in geo-distributed clouds," *J. Syst. Softw.*, vol. 187, no. C, may 2022. [Online]. Available: https://doi.org/10.1016/j.jss.2022.111227

[29] A. Choudhary, M. C. Govil, G. Singh, L. K. Awasthi, and E. S. Pilli, "Energy-aware scientific workflow scheduling in cloud environment," *Cluster Computing*, vol. 25, no. 6, p. 3845–3874, dec 2022. [Online]. Available: https://doi.org/10.1007/s10586-022-03613-3

[30] A. Taghinezhad-Niar, S. Pashazadeh, and J. Taheri, "Energy-efficient workflow scheduling with budget-deadline constraints for cloud," *Computing*, vol. 104, no. 3, p. 601–625, mar 2022. [Online]. Available: https://doi.org/10.1007/s00607-021-01030-9

[31] M. C. Cohen, P. W. Keller, V. Mirrokni, and M. Zadimoghaddam, "Overcommitment in cloud services: Bin packing with chance constraints," *Management Science*, vol. 65, no. 7, pp. 3255–3271, 2019.

[32] D. Medeiros, J. J. Williams, J. Wahlgren, L. S. M. Leite, and I. Peng, "Arc-v: Vertical resource adaptivity for hpc workloads in containerized environments," in *European Conference on Parallel Processing*. Springer, 2025, pp. 175–189.

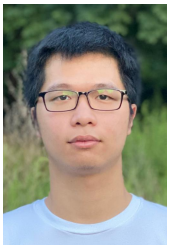
[33] G. P. Rodrigo, E. Elmroth, P.-O. Östberg, and L. Ramakrishnan, "Enabling workflow-aware scheduling on hpc systems," in *Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 3–14. [Online]. Available: https://doi.org/10.1145/3078597.3078604

[34] M. Tanash, B. Dunn, D. Andresen, W. Hsu, H. Yang, and A. Okanlawon, "Improving hpc system performance by predicting job resources via supervised machine learning," in *Proceedings of the Practice and Experience in Advanced Research Computing on Rise of the Machines (Learning)*, ser. PEARC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3332186.3333041

[35] C. Witt, J. van Santen, and U. Leser, "Learning low-wastage memory allocations for scientific workflows at icecube," in *2019 International Conference on High Performance Computing Simulation (HPCS)*, 2019, pp. 233–240.

[36] E. R. Rodrigues, R. L. F. Cunha, M. A. S. Netto, and M. Spriggs, "Helping hpc users specify job memory requirements via machine learning," in *2016 Third International Workshop on HPC User Support Tools (HUST)*, 2016, pp. 6–13.

[37] Y. Fan, Z. Lan, T. Childers, P. Rich, W. Allcock, and M. E. Papka, "Deep reinforcement agent for scheduling in hpc," 2021.



Thanh Son Phung received the BS degree in mathematics and computer science from Trinity College-Hartford and the MS degree in computer science and engineering from the University of Notre Dame. He is currently working toward the PhD degree with the Department of Computer Science and Engineering, the University of Notre Dame. His research focuses on improving resource efficiency and execution performance of large-scale workflow systems.

Douglas Thain is a professor in the Department of Computer Science and Engineering at the University of Notre Dame. He received the BS degree in Physics from the University of Minnesota - Twin Cities and the MS and PhD degrees in Computer Sciences from the University of Wisconsin - Madison. At Notre Dame, he leads a team researching and developing workflow systems, data management systems, and other technologies for large scale scientific computing.